

Study Guide — PDL Challenge 1

Neural Network-Based Function Upscaling: The Role of Activation Functions

How to use this guide

You are **not** expected to solve the challenge before the meeting — only to understand it. This guide gives you three things: (1) the minimum theory needed to read every sentence of the challenge PDF, (2) visual intuition for what the experiments will show (every figure here was produced with the challenge's own setup: $N = 512$, band-limit 50, seed 42), and (3) the vocabulary and questions to discuss it confidently.

The challenge in one paragraph

Generate smooth random functions (“band-limited”), keep only a few of their values (“samples”), and train neural networks to fill in the rest (“upsampling” / reconstruction). Classical signal processing solves this with linear formulas when enough samples exist. The challenge asks: what do *neural networks* do on this problem, and how does the answer depend on the **activation function**, the **architecture** (depth/width), and the **dataset size**?

There are two tasks, and they test different things:

- **Task 1 — fit one function.** A net with 1 input (position x) and 1 output (value y) memorises a single fixed function from M points. “Generalisation” = what the net draws *between* its training points.
- **Task 2 — learn the reconstruction operator.** A net with 100 inputs (the samples) and 512 outputs (the whole function) is trained on *many* random functions. “Generalisation” = reconstructing functions it has never seen.

Reading map

Challenge PDF section	What you need to know first	Covered in
Abstract + §1 Problem setting	Fourier transform, band-limited functions, conjugate symmetry	§1.1–1.3
§1.1 sample code	what each line does + one bug to fix	§1.4
§3 Architecture & constraints	neurons vs parameters, MSE loss, Adam optimiser	§2.1–2.2
§3.3 activation functions	the activation gallery and why the choice dominates	§2.3–2.4
§4 Task 1 / §4.1 Task 2	experiment playbooks, baselines, what to expect	§3–4
§5 Observe, observe, observe	hypothesis-driven experimentation	§5

1 The signal-processing half

1.1 A function as a recipe of sinusoids

The Discrete Fourier Transform (DFT) rewrites any 512-point function as a sum of sinusoids. Frequency “bin” k holds a complex coefficient $F[k]$ whose magnitude is the **amplitude** and whose angle is the **phase** of a sinusoid completing k cycles across the 512 samples. **Band-limited** simply means: all coefficients above some frequency are zero. The challenge generates functions with non-zero coefficients only for $k = 1 \dots 49$ — so nothing in the function oscillates faster than 49 cycles per 512 samples, which is why it looks smooth.

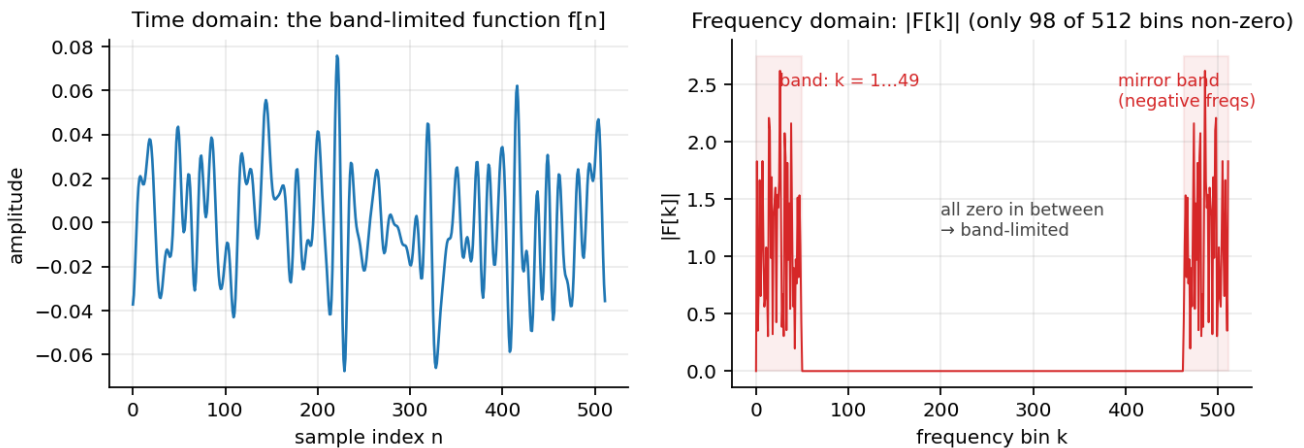


Fig 1 — The actual challenge function (seed 42). Left: time domain. Right: its spectrum — energy only in bins $1 \dots 49$ and their mirror images; every other bin is exactly zero.

1.2 Real functions $\Rightarrow$ conjugate symmetry

The generated function must be real-valued, and that forces the symmetry the PDF describes: $F[-k] = F[k]^*$ — magnitudes mirror (even symmetry), phases flip sign (odd symmetry). The code enforces exactly this when it copies the reversed conjugate coefficients into the negative-frequency slots.

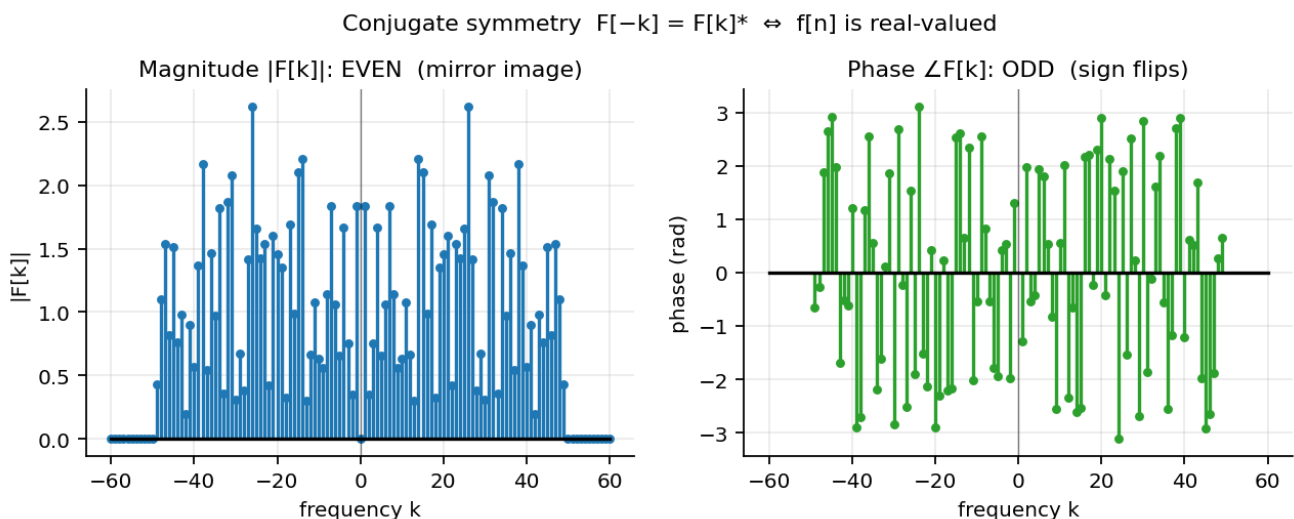


Fig 2 — Conjugate symmetry of the coefficients: magnitude is a mirror image around $k = 0$; phase is antisymmetric. This is what makes the inverse FFT come out real.

Key number: 98

Because of the symmetry, only the 49 coefficients $k = 1 \dots 49$ are free; each is one complex number = 2 real numbers. The DC term ($k = 0$) is set to zero. So **every function in this challenge is fully described by 98 real numbers**. Remember 98 — it explains half of the experimental results below.

1.3 Sampling: when is reconstruction even possible?

The Nyquist-Shannon theorem, in this discrete setting: a function with highest frequency 49 is pinned down exactly by any set of samples rich enough to separate 98 unknowns — e.g. ≥ 98 well-placed points (uniform spacing needs the sampling rate to exceed 2×49 cycles). With enough samples, reconstruction is pure linear algebra (classically: sinc interpolation). With fewer, high frequencies masquerade as low ones — **aliasing** — and the information is genuinely gone: no method, neural or not, can recover it.

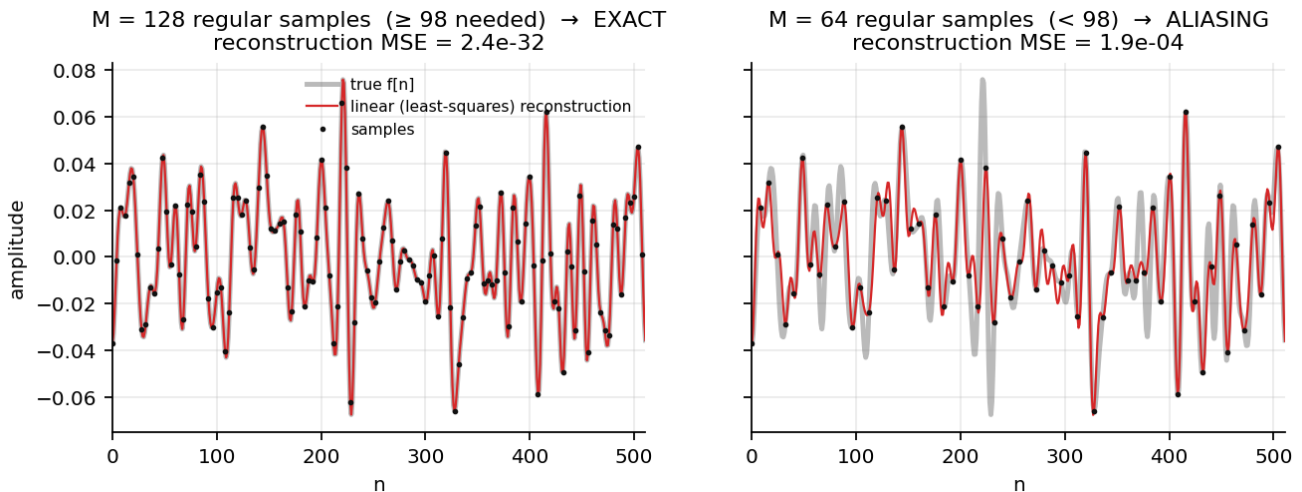


Fig 3 — Linear least-squares reconstruction on the known sinusoid basis. Left: 128 regular samples (> 98) recover the function to machine precision. Right: 64 samples (< 98) — the reconstruction aliases; error is unavoidable no matter the method.

1.4 The generator code, decoded

Line by line, the sample code: fix the seed (reproducibility) → draw random complex coefficients for bins 1...49 → mirror them conjugated into the negative bins (forces a real signal) → inverse FFT → take .real (kills tiny numerical imaginary residue) → plot.

Two practical gotchas

(1) `np.complex` was removed in NumPy ≥ 1.24 — write `dtype=complex` instead. (2) The generated values are small ($\approx \pm 0.07$, since `ifft` divides by N). Standardise y — and rescale x from 1...512 into $[-1, 1]$ — before training, or optimisation will be needlessly slow.

2 The deep-learning half

2.1 MLPs, and what the “neuron budget” means

Both tasks use plain fully-connected networks (MLPs): alternating linear layers and element-wise activations. The budget counts **hidden neurons (units), not parameters (weights)**. A 3×128 network uses 384 neurons — far under Task 1's 10,000 — but has $\approx 33,000$ parameters (each layer contributes $(n_{\text{in}} + 1) \cdot n_{\text{out}}$ weights). Task 2's million-neuron budget is generous: 2×2048 uses only 4,096 neurons, yet already ≈ 5.5 million parameters. Depth vs width at a fixed budget is precisely one of the experiments.

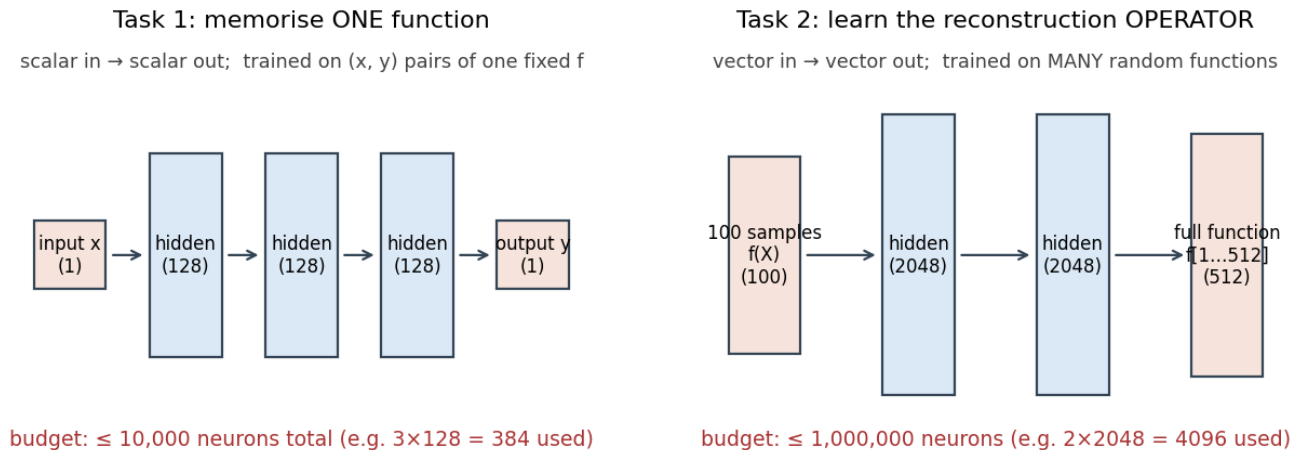


Fig 4 — The two tasks as diagrams. Task 1: scalar-to-scalar curve fitting on one function. Task 2: vector-to-vector operator learning across many functions.

2.2 Training: MSE + Adam until convergence

Both tasks minimise the mean squared error, $\text{MSE} = (1/M) \sum (\text{net}(x_i) - y_i)^2$, with the Adam optimiser (gradient descent with per-parameter adaptive step sizes — the robust default). “Until convergence” means: train until the loss curve flattens, not a fixed small number of epochs.

Practical training tips (they will save your experiments)

- Normalise x to $[-1, 1]$; standardise y to zero mean, unit variance.
- Task 1 datasets are tiny — use full-batch gradients; expect 10,000+ iterations.
- Learning rate $\approx 10^{-3}$ (sine/SIREN nets prefer $\approx 10^{-4}$); decay it near the end.
- Always repeat with ≥ 3 random seeds — single runs can mislead.
- Log train MSE *and* full-grid MSE separately; the gap is the interesting quantity.

2.3 The activation function gallery

The activation is the only non-linear ingredient — without it a deep stack collapses into one linear map. Its shape (smoothness, saturation, periodicity) becomes the “texture” of everything the network draws.

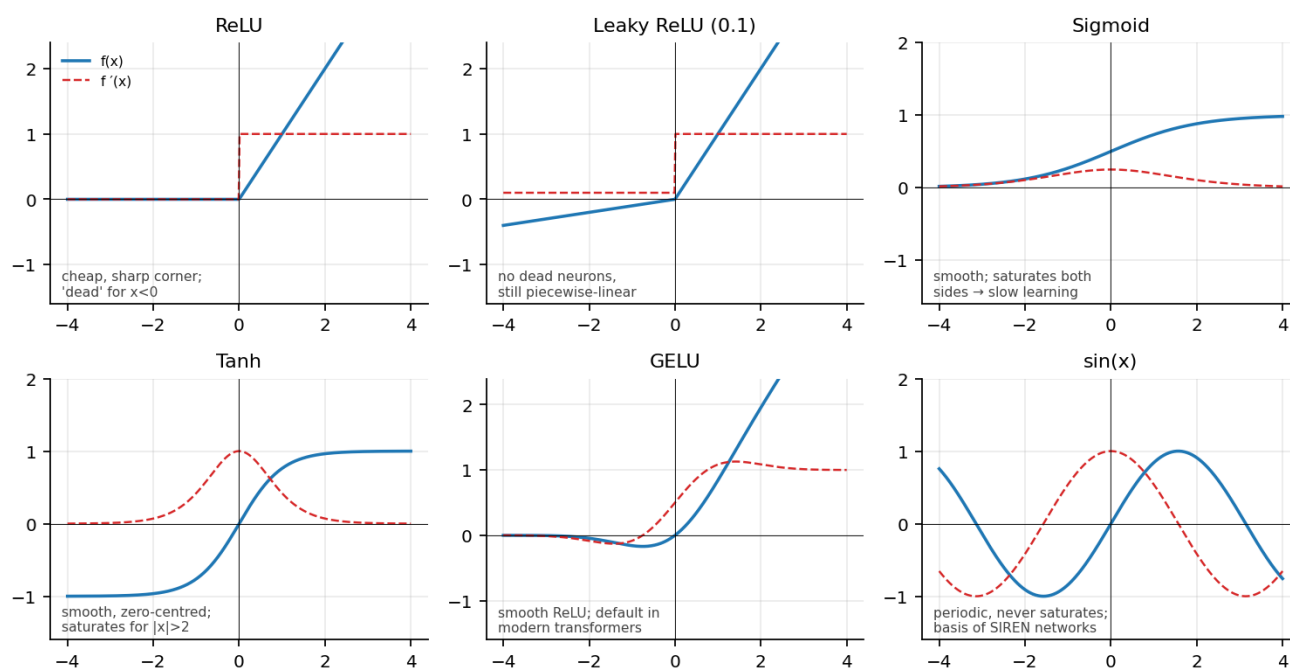


Fig 5 — Six activations and their derivatives. Solid: $f(x)$; dashed: $f'(x)$. The derivative determines both gradient flow during training and the smoothness of the fitted curve.

Activation	Smooth?	Saturates?	Known issues	When it shines
ReLU	no (corner at 0)	no (for $x > 0$)	dead neurons; piecewise-linear output	deep nets, classification, speed
Leaky ReLU	no	no	still piecewise-linear	like ReLU, avoids dead units
Sigmoid	yes	both sides	vanishing gradients; not zero-centred	output layers for probabilities
Tanh	yes	both sides	vanishing gradients when deep	smooth low-frequency regression
GELU	yes	left side	slightly costlier	transformers; smooth ReLU substitute
sin (SIREN)	yes	never	needs special init + smaller lr	signals, images, implicit fields — this challenge

2.4 Why the activation dominates this particular problem

Three ideas explain most of what you will observe:

- **A ReLU network is a piecewise-linear function.** Compositions of linear maps and $\max(0, \cdot)$ can only produce straight-line segments. Fitting a wave means approximating curves with many tiny segments, and between training points the net draws — literally — straight lines.
- **Spectral bias.** Gradient descent on standard MLPs fits low frequencies first and high frequencies reluctantly or never. This target has energy up to $k = 49$, so ReLU/tanh nets capture the slow envelope and miss the fast wiggles (watch the band edge in Fig 7).
- **Periodic activations build sinusoids natively.** A sine-activated net (SIREN) starts from the “correct basis” for band-limited signals — matching the model family to the signal class.

Same 100 training points, same 1→128→128→128→1 net — only the activation changes
(for reference: band-limit-aware least squares on the SAME points is exact, $\text{MSE} \approx 4\text{e-}18$)

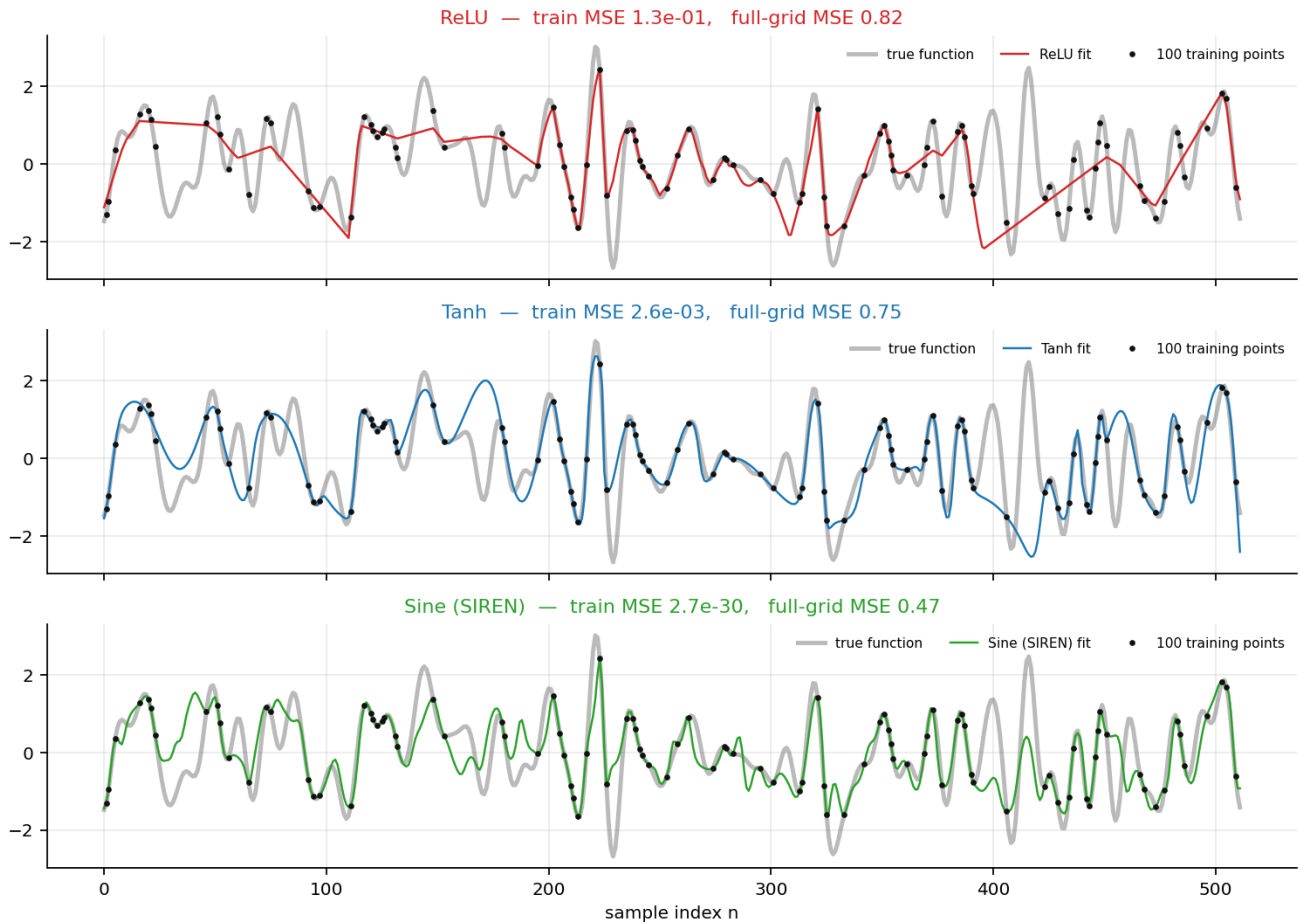


Fig 6 — The core experiment of the whole challenge, run for you once: same 100 points, same 1→128→128→128→1 architecture, Adam until convergence; only the activation changes. ReLU (top) cannot even fit the training points (train MSE 0.13) and draws straight lines between them. Tanh (middle) memorises the points (2.6×10^{-3}) but over-smooths between them. Sine (bottom) memorises to 10^{-30} and interpolates best.

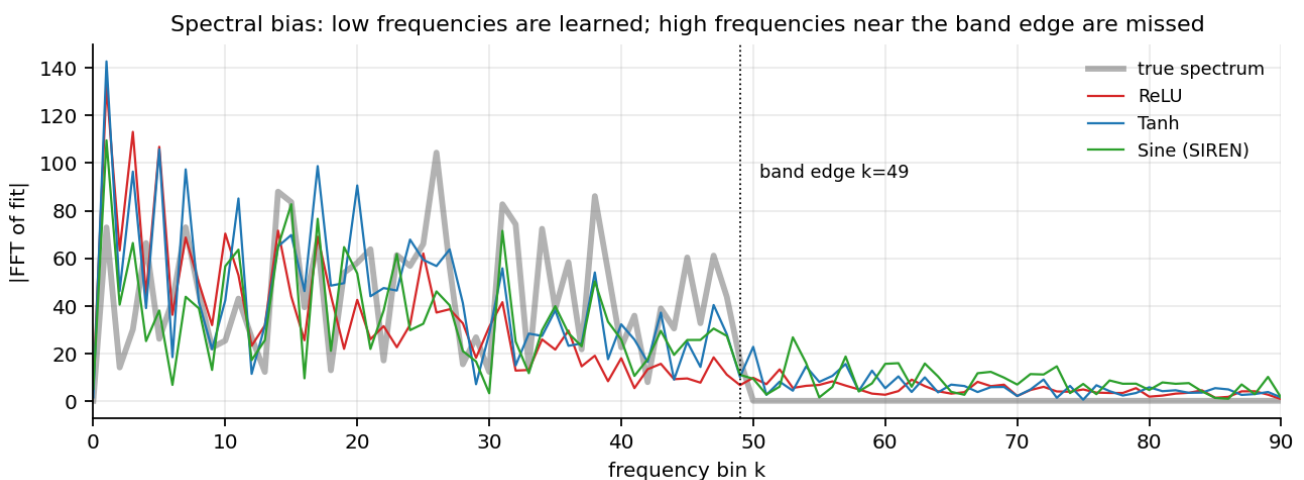


Fig 7 — The same three fits viewed in the frequency domain. All capture low frequencies; ReLU loses the most near the band edge ($k \approx 30-49$); the sine net tracks the spectrum furthest. This is spectral bias made visible.

The punchline (worth saying at the meeting)

On the *same* 100 points, band-limit-aware least squares (§1.3) reconstructs the function **exactly** ($\text{MSE} \approx 10^{-18}$), while the best neural net reaches ≈ 0.47 . Between data points, what a model draws is decided by its **inductive bias** — what its architecture and activation “naturally do” — not by the data, which is identical in all cases. The challenge is engineered so you discover this.

3 Task 1 playbook — memorise one function

Fix one random band-limited function. Dataset = M of its 512 (x, y) pairs. Train, then evaluate on **all 512 grid positions**. Two numbers per run: train MSE (did it memorise?) and full-grid MSE (what did it draw in between?). Their gap is the interpolation quality — Task 1's real subject.

Effect of dataset size M

The information threshold from §1 bites here: below 98 samples *no* method can identify the function, so the net fills gaps with its own prior — tanh draws calm flat curves, ReLU draws polylines. Above the threshold, fits snap into place rapidly:

Tanh network, identical architecture — only the dataset size M changes

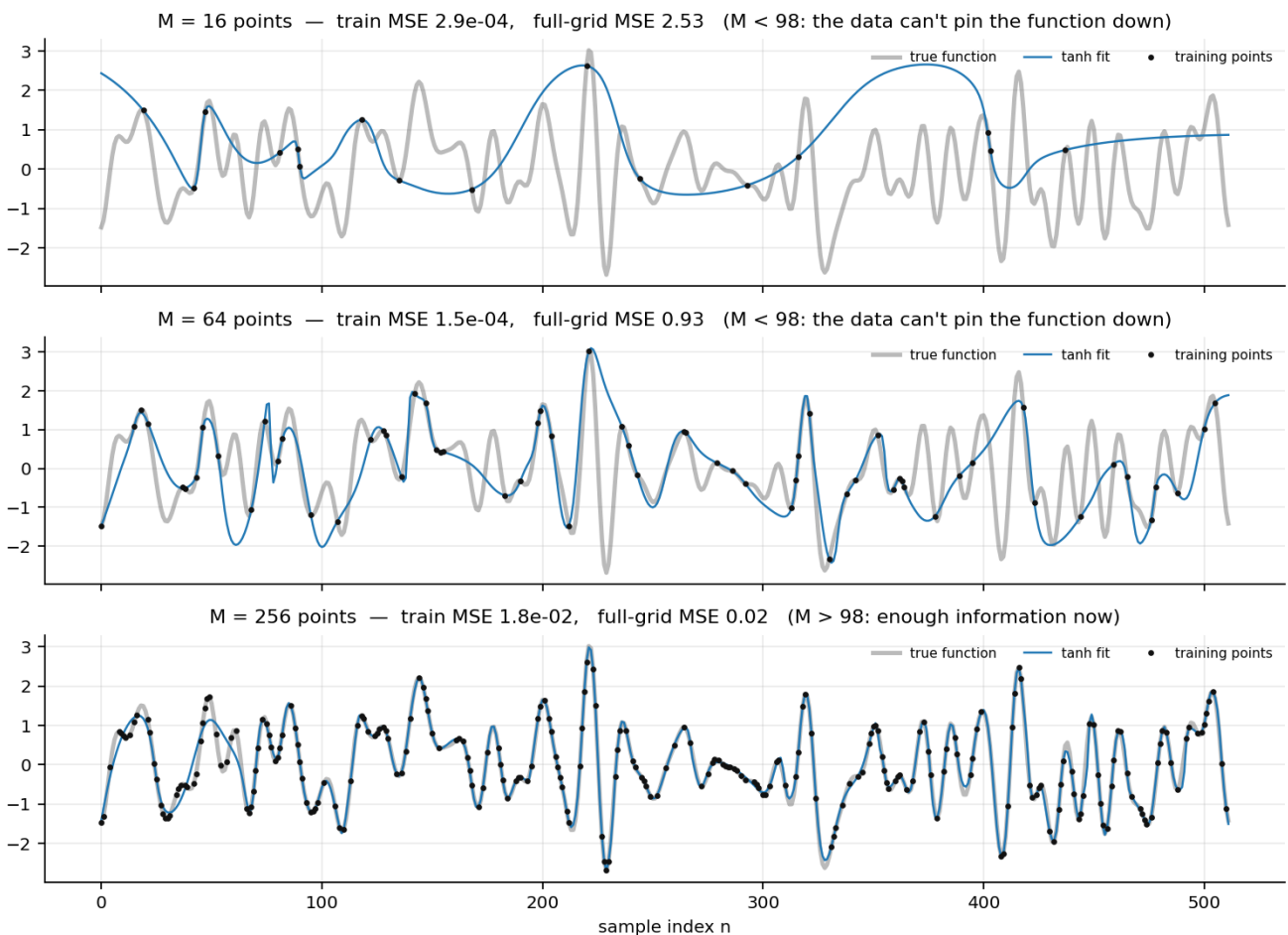


Fig 8 — Tanh net, identical architecture and training, only M changes. $M = 16$: full-grid MSE 2.5 — worse than predicting the mean (!) despite train MSE 3×10^{-4} . $M = 64$: envelope emerges (0.93). $M = 256$: nearly perfect (0.02). Note how the improvement is not gradual — it accelerates once M crosses the ≈ 98 -sample information threshold.

Effect of where the samples sit (the set X)

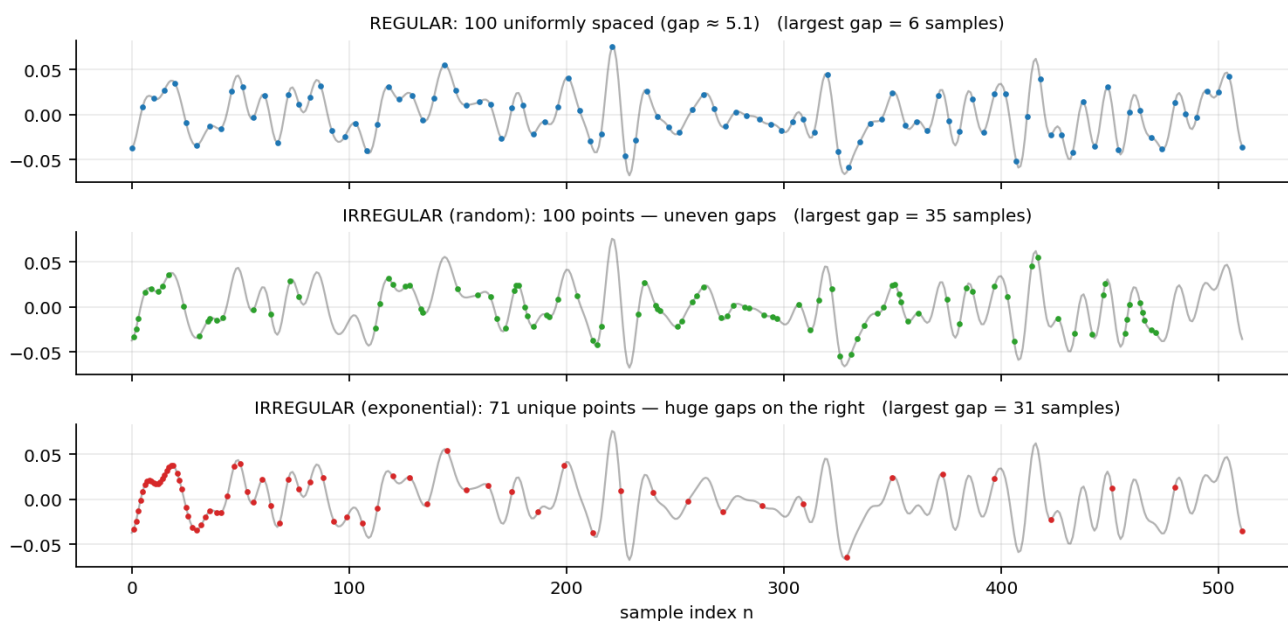


Fig 9 — Three choices of 100 sample positions. Regular spacing: largest gap 6. Random: largest gap 35. Exponential: rounding to integers collapses 100 requested points into 71 unique ones, with 31-sample deserts on the right — count your unique indexes! Expect the local error to track the local gap size.

Suggested experiment grid

Vary	Values to try	Hold fixed	Plot
dataset size M	10, 25, 50, 100, 200, 400	3×128 net, activation, random X, 3 seeds	full-grid MSE vs M (log-log) — look for the elbow near 98
activation	relu, tanh, sine (+ gelu)	M = 100, architecture, X	overlay fits (as Fig 6); bar chart of MSEs
depth × width (equal budget)	1×384, 3×128, 6×64	M, activation, X	MSE vs depth; note trainability of deep tanh
sampling pattern X	regular / random / exponential	M = 100, net	error vs position, with sample locations marked

What you should expect (hypotheses to verify, not trust)

- ReLU shows the largest full-grid error and visible straight segments; sine the smallest; tanh in between.
- Full-grid MSE vs M drops steeply around $M \approx 100$ and then saturates at a floor set by the activation.
- At equal neuron budget, moderate depth beats both extremes: 1 wide layer underfits curvature; very deep tanh gets hard to optimise.
- Errors concentrate in sampling gaps; exponential spacing fails on the sparse side even for large M.

4 Task 2 playbook — learn the reconstruction operator

Now the network never sees x at all. Input: the vector of 100 sampled values $f(X)$; output: all 512 values of f . Training set: M different random functions. The net is learning the **operator** “samples $\rightarrow$ function”, i.e. exactly what sinc interpolation does in classical DSP. Crucially, test it on **new** random functions the net never saw — this is true generalisation across a function class, unlike Task 1's within-one-function interpolation.

The insight that reframes the whole task

Reconstruction from samples is a **linear** map (§1.3): each output value is a fixed weighted sum of the 100 inputs. With regular X , 100 samples $\geq$ 98 degrees of freedom, so **a single linear layer with no activation can already be essentially exact**. Run that baseline first. The scientific question then becomes: do nonlinear activations help, hurt, or merely need more data to match a linear model — and does the answer flip when X is irregular (ill-conditioned) or samples are too few?

What changes compared to Task 1

- **Generalisation axis:** across functions, not across positions. Always keep a validation set of fresh functions (different seed).
- **Dataset size M now counts functions.** The input vectors span a 98-dimensional subspace, so interesting behaviour starts near $M \approx 100$; sweep $M \in \{50, 100, 200, 500, 2000, 10000\}$ and find the plateau.
- **Architecture:** the budget (10^6 neurons) is not the binding constraint — parameters and memory are. 2×2048 ReLU (≈ 5.5 M parameters) is a solid starting point; mini-batches of 128–512 with Adam.
- **Choice of X :** with irregular or exponential spacing the linear problem becomes ill-conditioned (and with the naive exponential grid, only 71 unique indexes — fewer than 98 — so even perfect linear reconstruction is impossible). Plot per-position validation error with sample locations marked: errors pool in the gaps.

Suggested experiment grid

Vary	Values to try	Hold fixed	Plot
model class	linear (no activation), relu, tanh	$M = 2000$, regular X	val MSE bar chart — can anything beat linear?
dataset size M	50 ... 10000	2×2048 relu, regular X	val MSE vs M (log-log): decay then plateau
sampling set X	regular / random / exponential	$M = 2000$, model	per-position error vs sample locations
depth $\times$ width	1×4096 , 2×2048 , 4×1024	M , X	val MSE + training time

What you should expect

- With regular X the linear model is near-exact; nonlinear nets approach it given enough M but rarely beat it on clean data.
- Validation error falls roughly as a power law in M , then plateaus once the operator is identified.
- Irregular X : errors grow inside large gaps; nonlinearity + more data help somewhat, but conditioning rules.
- If you add noise to the samples (optional experiment), nonlinear nets can start to win — priors matter once information is imperfect.

5 Run experiments like a scientist (challenge §5)

The challenge's final section is really the grading rubric: it wants hypothesis-driven experiments, not a pile of loss curves. For each claim: define variables, control confounders, span a wide range of conditions, replicate over seeds, and only then conclude. Four ready-to-test hypotheses:

Hypothesis	Independent variable	Control (hold fixed)	Evidence to collect
H1: full-grid error decays as a power law in M until an information/capacity floor	M	architecture, activation, X, seeds	log-log MSE vs M; slope + elbow location (≈ 98 ?)
H2: smooth/periodic activations beat ReLU on band-limited targets at equal budget	activation	M, architecture, X	MSE table + fitted curves + spectra (Figs 6–7)
H3 (Task 2): a linear model matches nonlinear nets when X is regular and clean	model class	M, X	val MSE linear vs relu vs tanh across M
H4: local error is predicted by local sampling gap, not by M alone	X pattern	M, model	error-vs-position overlaid with gap sizes

Experiment hygiene checklist

- Fix and record every seed (data, init, batch order).
- Same convergence criterion for every run.
- ≥ 3 seeds per configuration; report mean $\pm$ std.
- Change one variable at a time.
- Save loss curves, not just final numbers.
- Normalise identically across runs — otherwise MSEs are not comparable.

6 Self-test before the meeting

Q. Why are the Fourier coefficients conjugate-symmetric?

A. Because the time-domain function is real-valued; realness $\Leftrightarrow F[-k] = F[k]^*$.

Q. How many real numbers fully describe one generated function?

A. 98 (49 free complex coefficients $\times 2$; DC term is zero).

Q. Roughly how many samples does perfect classical reconstruction need?

A. ≥ 98 well-placed samples (uniform sampling above the Nyquist rate for $k_{\max} = 49$). Below that: aliasing, information lost.

Q. Why is a ReLU network's output piecewise linear?

A. It composes linear maps with $\max(0, \cdot)$; every region of input space gets one affine formula.

Q. What is spectral bias?

A. Gradient-trained MLPs learn low frequencies first; high-frequency detail comes late or never.

Q. How is generalisation measured differently in the two tasks?

A. Task 1: unseen positions of the same function (full-grid MSE). Task 2: entirely unseen functions (validation set).

Q. Why could a purely linear network solve Task 2 with regular X?

A. Reconstruction from ≥ 98 informative samples is a linear operator — 100 regular samples suffice.

Q. Why repeat runs with several seeds?

A. Initialisation and data draws add variance; conclusions need mean $\pm$ std, not one lucky run.

Mini-glossary

DFT bin: one frequency slot of the discrete Fourier transform. **Band-limit**: highest non-zero frequency. **Nyquist rate**: minimum sampling rate ($2\times$ highest frequency) for exact linear recovery. **Aliasing**: high frequencies disguising as low ones after undersampling. **Sinc interpolation**: the classical linear reconstruction formula. **Neuron vs parameter**: hidden unit vs trainable weight. **Epoch/iteration**: pass over data / single gradient step. **Adam**: adaptive-step gradient optimiser. **Overfitting**: excellent on training data, poor elsewhere — *deliberate* in Task 1. **Generalisation gap**: train-vs-test error difference. **Inductive bias**: what a model family does where data is silent. **Spectral bias**: see self-test. **SIREN**: MLP with sine activations and matched initialisation (Sitzmann et al., 2020).

Good questions to ask at the meeting

- Is the neuron budget counting hidden units only, and are there constraints on parameters or training time?
- For Task 1, should the evaluation grid be exactly the 512 integer positions, or also off-grid x ?
- What convergence criterion should we standardise on, so results are comparable between students?
- May we add a noisy-samples variant in Task 2 to test whether nonlinearity starts to pay off?

Appendix A — Task 1 starter code (task1_starter.py)

Also provided as a separate runnable file. PyTorch; sweeps activations at $M = 100$; TODOs mark the remaining experiment axes.

```
"""
PDL Challenge 1 – Task 1 starter: a DNN memorises ONE band-limited function.
Requires: pip install torch numpy matplotlib
"""

import numpy as np
import torch
import torch.nn as nn
import matplotlib.pyplot as plt

# ----- 1. Generate the target band-limited function -----
def bandlimited_function(N=512, band_limit=50, seed=42):
    rng = np.random.RandomState(seed)
    c = np.zeros(N, dtype=complex)
    # np.complex is removed in NumPy>=1.24
    c[1:band_limit] = rng.randn(band_limit - 1) + 1j * rng.randn(band_limit - 1)
    c[-(band_limit - 1):] = c[1:band_limit][::-1].conj() # conjugate symmetry -> real signal
    return np.fft.ifft(c).real
    # shape (N,)

N = 512
f = bandlimited_function(N)

# ----- 2. Dataset: M points of (x, y) -----
def make_dataset(f, M, mode="random", seed=0):
    N = len(f)
    if mode == "regular":
        idx = np.linspace(0, N - 1, M).astype(int)
    elif mode == "random":
        idx = np.sort(np.random.default_rng(seed).choice(N, M, replace=False))
    elif mode == "exponential":
        idx = np.unique(np.round(np.logspace(0, np.log10(N - 1), M)).astype(int))
    x = 2 * idx / (N - 1) - 1 # ALWAYS normalise x to [-1, 1]
    y = (f[idx] - f.mean()) / f.std() # and standardise y
    return (torch.tensor(x, dtype=torch.float32).unsqueeze(1),
            torch.tensor(y, dtype=torch.float32).unsqueeze(1), idx)

# ----- 3. Model builder (stay under 10,000 neurons!) -----
class Sine(nn.Module):
    # SIREN-style activation
    def __init__(self, w0=30.0): super().__init__(); self.w0 = w0
    def forward(self, x): return torch.sin(self.w0 * x)

def make_mlp(depth=3, width=128, act="tanh"):
    acts = {"relu": nn.ReLU, "tanh": nn.Tanh, "sigmoid": nn.Sigmoid,
            "gelu": nn.GELU, "sine": Sine}
    layers, d_in = [], 1
    for _ in range(depth):
        layers += [nn.Linear(d_in, width), acts[act]()
        d_in = width
    layers += [nn.Linear(d_in, 1)]
    net = nn.Sequential(*layers)
    n_neurons = depth * width # "neurons" = hidden units
    assert n_neurons <= 10_000, f"budget exceeded: {n_neurons}"
    print(f"{depth}x{width} {act}: {n_neurons} neurons, "
          f"{sum(p.numel() for p in net.parameters())} parameters")
    return net

# ----- 4. Train until convergence -----
def train(net, X, Y, lr=1e-3, max_iters=30_000, tol=1e-7):
    opt = torch.optim.Adam(net.parameters(), lr=lr)
    sched = torch.optim.lr_scheduler.ReduceLROnPlateau(opt, factor=0.5, patience=2000)
    prev = np.inf
    for i in range(max_iters):
        opt.zero_grad()
        loss = nn.functional.mse_loss(net(X), Y)
        loss.backward(); opt.step(); sched.step(loss)
        if i % 2000 == 0:
            print(f" iter {i:6d} train MSE {loss.item():.3e}")
            if abs(prev - loss.item()) < tol: break
            prev = loss.item()
    return loss.item()

# ----- 5. Evaluate on the FULL grid (this is the real test) -----
def evaluate(net, f):
    N = len(f)
    x_full = torch.linspace(-1, 1, N).unsqueeze(1)
    with torch.no_grad():
        pred = net(x_full).squeeze().numpy()
    y_full = (f - f.mean()) / f.std()
    return pred, float(np.mean((pred - y_full) ** 2))

# ----- 6. Experiment loop -----
if __name__ == "__main__":
```

```

torch.manual_seed(0)
results = {}
M = 100
X, Y, idx = make_dataset(f, M, mode="random")
for act in ["relu", "tanh", "sine"]:
    net = make_mlp(depth=3, width=128, act=act)
    tr = train(net, X, Y, lr=1e-4 if act == "sine" else 1e-3)
    pred, test_mse = evaluate(net, f)
    results[act] = (pred, tr, test_mse)
    print(f"==> {act}: train {tr:.2e}    full-grid {test_mse:.3f}\n")

fig, axes = plt.subplots(3, 1, figsize=(10, 8), sharex=True)
y_full = (f - f.mean()) / f.std()
for ax, (act, (pred, tr, te)) in zip(axes, results.items()):
    ax.plot(y_full, "gray", lw=2, alpha=0.5, label="true")
    ax.plot(pred, lw=1, label=f"{act} (MSE {te:.3f})")
    ax.plot(idx, y_full[idx], "k.", ms=4)
    ax.legend()
plt.savefig("task1_results.png", dpi=150); plt.show()

# TODO for your experiments:
# - sweep M in [10, 25, 50, 100, 200, 400]; plot full-grid MSE vs M (log-log)
# - sweep depth/width at fixed neuron budget (e.g. 1x384 vs 3x128 vs 6x64)
# - compare mode="regular" vs "random" vs "exponential"
# - repeat each config over >=3 seeds; report mean +/- std

```

Appendix B — Task 2 starter code (task2_starter.py)

Trains the operator net on M random functions and validates on unseen ones; the linear baseline is one argument away (act="linear").

```
"""
PDL Challenge 1 — Task 2 starter: a DNN learns to reconstruct ALL band-limited
functions from 100 samples. Input: 100 values f(X). Output: all 512 values.
Requires: pip install torch numpy matplotlib
"""

import numpy as np
import torch
import torch.nn as nn
import matplotlib.pyplot as plt

N, BAND = 512, 50

# ----- 1. Generate M random band-limited functions -----
def random_functions(M, seed=0):
    rng = np.random.RandomState(seed)
    F = np.zeros((M, N))
    for m in range(M):
        c = np.zeros(N, dtype=complex)
        c[1:BAND] = rng.randn(BAND - 1) + 1j * rng.randn(BAND - 1)
        c[-(BAND - 1):] = c[1:BAND][::-1].conj()
        F[m] = np.fft.ifft(c).real
    return F # (M, 512)

# ----- 2. Choose the sampling set X (100 indexes) -----
def sampling_set(kind="regular", n_samples=100, seed=0):
    if kind == "regular":
        return np.linspace(0, N - 1, n_samples).astype(int)
    if kind == "random":
        return np.sort(np.random.default_rng(seed).choice(N, n_samples, replace=False))
    if kind == "exponential":
        return np.unique(np.round(np.logspace(0, np.log10(N - 1), n_samples)).astype(int))

# ----- 3. Dataset: predictor f(X) (M,100) -> label f (M,512) -----
def make_dataset(M, kind="regular", seed=0):
    F = random_functions(M, seed)
    Xset = sampling_set(kind)
    scale = F.std() # one global scale
    A = torch.tensor(F[:, Xset] / scale, dtype=torch.float32)
    B = torch.tensor(F / scale, dtype=torch.float32)
    return A, B, Xset, scale

# ----- 4. Model (stay under 1,000,000 neurons) -----
def make_model(depth=2, width=2048, act="relu", n_in=100):
    acts = {"relu": nn.ReLU, "tanh": nn.Tanh, "gelu": nn.GELU, "linear": None}
    layers, d = [], n_in
    for _ in range(depth if act != "linear" else 0):
        layers += [nn.Linear(d, width), acts[act]() if act != "linear" else None]
        d = width
    layers += [nn.Linear(d, N)]
    net = nn.Sequential(*layers)
    print(f"{act} {depth}x{width}: {depth*width} neurons, "
          f"{sum(p.numel() for p in net.parameters()):,} parameters")
    return net

# ----- 5. Train / evaluate -----
def train(net, A, B, Aval, Bval, lr=1e-3, epochs=3000, batch=256):
    opt = torch.optim.Adam(net.parameters(), lr=lr)
    M = A.shape[0]
    for ep in range(epochs):
        perm = torch.randperm(M)
        for i in range(0, M, batch):
            j = perm[i:i + batch]
            opt.zero_grad()
            loss = nn.functional.mse_loss(net(A[j]), B[j])
            loss.backward(); opt.step()
        if ep % 200 == 0:
            with torch.no_grad():
                vl = nn.functional.mse_loss(net(Aval), Bval).item()
                print(f" epoch {ep:5d} train {loss.item():.3e} VAL {vl:.3e}")
    return net

if __name__ == "__main__":
    torch.manual_seed(0)
    M = 2000 # sweep this: 100 ... 20000
    A, B, Xset, scale = make_dataset(M, kind="regular", seed=0)
    Aval, Bval, _, _ = make_dataset(500, kind="regular", seed=999) # NEW functions!

    net = make_model(depth=2, width=2048, act="relu")
    train(net, A, B, Aval, Bval)
    # --- linear baseline: 100 regular samples >= 98 degrees of freedom => exact ---
```

```

# a purely linear model (act="linear") can reach ~0 error; can your DNN beat it?

with torch.no_grad():
    rec = net(Aval[:3]).numpy() * scale
fig, axes = plt.subplots(3, 1, figsize=(10, 7), sharex=True)
for k, ax in enumerate(axes):
    ax.plot(Bval[k].numpy() * scale, "gray", lw=2, alpha=0.5, label="true (unseen function)")
    ax.plot(rec[k], "C3", lw=1, label="DNN reconstruction")
    ax.plot(Xset, Aval[k].numpy() * scale, "k.", ms=4, label="100 given samples")
    ax.legend()
plt.savefig("task2_results.png", dpi=150); plt.show()

# TODO for your experiments:
# - sweep dataset size M; plot VAL MSE vs M (log-log). Where does it plateau?
# - act="linear" vs "relu" vs "tanh": does nonlinearity even help here? why?
# - kind="regular" vs "random" vs "exponential": where do errors concentrate?
#   (plot per-position error averaged over the val set vs sample locations)
# - depth vs width under the same neuron budget

```